SKT SOFTWARE SOLUTION	**SKT Software Solution**	**CÓDIGO:** DI-GDR-RGI-001
	PROCESO: Gestión de Proyectos - Ciclo PDCA	**VERSIÓN:** 1.0
	Informe de Mejora Continua	**ELABORACIÓN:** 21/05/2026
	Matrices PDCA del Proyecto Centro Médico	**ÚLTIMA REVISIÓN:** 21/05/2026

1. Título

Informe de Aplicación del Ciclo de Mejora Continua PDCA en el Proyecto de Gestión del Centro Médico LUPSI

2. Introducción

El presente informe documenta la implementación del ciclo de mejora continua PDCA (Planificar-Hacer-Verificar-Actuar) en el desarrollo del sistema de gestión para un centro médico, desarrollado por SKT Software Solution en el marco de la asignatura Gestión de Proyectos de la carrera de Ingeniería en Software. El propósito es evidenciar cómo se han identificado y corregido desviaciones en cada fase del proyecto (Planificación y requisitos, Arquitectura y Diseño UI/UX, Desarrollo), garantizando la trazabilidad de las acciones correctivas y el cumplimiento de los estándares de calidad.

El alcance del informe abarca desde el levantamiento inicial de requisitos hasta la fase de desarrollo, incluyendo la integración frontend-backend. Las matrices PDCA completas se adjuntan como imágenes en los anexos, mientras que en el cuerpo del documento se analiza en detalle cada ciclo.

3. Objetivo

Exponer de manera sistemática la aplicación del ciclo PDCA en cada fase del proyecto de centro médico, detallando los problemas identificados, las acciones planificadas y ejecutadas, los mecanismos de verificación y las acciones de estandarización o mejora, con el fin de consolidar un proceso de gestión basado en la mejora continua.

4. Definiciones

- **PDCA:** Ciclo de mejora continua compuesto por Planificar (establecer objetivos y procesos), Hacer (ejecutar lo planificado), Verificar (medir y evaluar resultados) y Actuar (ajustar y estandarizar mejoras).
- **Product Backlog:** Lista priorizada de requisitos funcionales y no funcionales, historias de usuario y tareas técnicas que guían el desarrollo del producto.

- **GitFlow:** Modelo de branching para Git que define ramas específicas (main, develop, feature, release, hotfix) para gestionar el desarrollo colaborativo y los lanzamientos.
- **Row Level Security (RLS):** Mecanismo de seguridad a nivel de base de datos que restringe el acceso a filas según el rol del usuario.
- **3FN (Tercera Forma Normal):** Nivel de normalización de bases de datos que elimina dependencias transitivas, garantizando integridad y reduciendo redundancia.

5. Desarrollo y Contenido

5.1. Metodología de trabajo con PDCA

El equipo de proyecto adoptó el ciclo PDCA como herramienta central de aseguramiento de la calidad. Para cada fase se elaboró una matriz que documenta:

- **Planificar (P):** Identificación del problema o punto de mejora, definición de objetivos y plan de acción.
- **Hacer (D):** Ejecución de las actividades planificadas, asignación de responsables y registro de evidencias.
- **Verificar (C):** Revisión de resultados mediante indicadores, comparación con lo esperado y retroalimentación.
- **Actuar (A):** Estandarización de buenas prácticas o ajuste de procesos para futuros ciclos.

A continuación se desglosa la aplicación concreta en cada fase del proyecto.

5.2. Fase 1: Planificación y requisitos

5.2.1. Contexto y problema identificado

Durante la fase inicial se detectó que el levantamiento de requisitos carecía de una estructura formal: no existía documentación unificada, los procesos médicos no eran comprendidos en su totalidad por el equipo técnico y las funcionalidades no estaban priorizadas. Esto generaba riesgo de omisiones y retrabajos.

5.2.2. Ciclo PDCA aplicado

- **Planificar:** El gestor del proyecto (Angel Ayuquina) definió un plan de mejora que incluía la elaboración de una plantilla estandarizada para requisitos, la programación de entrevistas con el profesional médico y la creación de un cronograma de validaciones. Se estableció como evidencia la documentación completa de requisitos y un cronograma aprobado en Trello.
- **Hacer:** Se ejecutaron entrevistas estructuradas con la Dra. Luz Marina Saenz (especialista del centro médico). Se recopiló información sobre el flujo actual de atención, puntos críticos y necesidades específicas. Se generaron actas de cada entrevista.
- **Verificar:** Los requisitos obtenidos fueron validados mediante la construcción de historias de usuario, revisadas por el PM y el equipo técnico. Se verificó la coherencia con los objetivos del proyecto y se corrigieron ambigüedades. La validación se consideró exitosa al obtener la aprobación del cliente sobre las historias.
- **Actuar:** Con los requisitos validados se construyó el Product Backlog, priorizando funcionalidades según su valor para el negocio y dependencias técnicas. Este backlog se transfirió a la herramienta Trello, estableciendo un proceso de actualización continua. Se definió que para futuras fases se repetiría el mismo esquema de entrevista+validación+backlog.

5.2.3. Resultados y evidencias

Se obtuvo un Product Backlog priorizado con 28 historias de usuario, un cronograma base aprobado y un repositorio documental en Trello que sirvió como fuente única de verdad. No se registraron requisitos omitidos en fases posteriores gracias a este proceso.

5.3. Fase 2: Arquitectura y Diseño UI/UX

5.3.1. Contexto y problema identificado

En la fase de diseño se evidenció la ausencia de un modelo de persistencia relacional normalizado y de prototipos de interfaces validados con el usuario final. Esto representaba un alto riesgo de redundancia de datos, retrasos por falta de un entorno

colaborativo estándar (Git) y posible rechazo por parte del personal médico al no involucrarlos en el diseño de la interfaz.

5.3.2. Ciclo PDCA aplicado

- **Planificar:** El equipo de arquitectura (Daniel Luisa), base de datos (Sebastián Ortiz) y UX/UI (Alex Guachi), coordinados por el PM, definieron un plan que incluía: (1) diseño del diagrama Entidad-Relación en 3FN, (2) elaboración de prototipos interactivos de alta fidelidad, (3) definición de la arquitectura tecnológica (Angular PWA, NestJS, Supabase) y (4) configuración de GitFlow y entornos Docker. Se estableció como evidencia la aprobación docente de los requisitos y la especificación tecnológica.

- **Hacer:** Se construyó el diagrama ER conceptual y lógico normalizado a 3FN, generando el script DDL completo (tablas de citas, pacientes, doctores, etc.). Se diseñaron prototipos navegables en Draw.io y Lucidchart, y se inicializó el repositorio GitHub con ramas bajo GitFlow. Además, se crearon contenedores Docker para unificar los entornos de desarrollo.

- **Verificar:** Se realizaron ajustes visuales a los prototipos basados en retroalimentación directa de la Dra. Saenz. Se implementaron reglas de integración colaborativa: Pull Requests obligatorios y revisiones de código. Las sentencias DDL fueron exportadas con políticas de Row Level Security (RLS) y se afinó la especificación de APIs RESTful. La verificación se completó con un flujo GitFlow sin conflictos y una base de datos lista para despliegue.

- **Actuar:** Se llevaron a cabo sesiones de validación de usabilidad con la Dra. Saenz y el patrocinador académico Ing. Paulo Torres. Se auditó el modelo de datos en Draw.io para asegurar integridad referencial y 3FN. El informe de modelado y arquitectura (código MI-DES-ARQ-002) fue firmado digitalmente por todo el equipo el 17/03/2026. Como acción de estandarización, se creó una guía de buenas prácticas para futuros diseños de bases de datos y prototipado.

5.3.3. Resultados y evidencias

Se obtuvo un modelo de datos normalizado y documentado, prototipos de alta fidelidad aprobados por el cliente, un repositorio colaborativo funcional y entornos estandarizados. La validación temprana evitó rediseños costosos en fases de desarrollo.

5.4. Fase 3: Desarrollo

5.4.1. Contexto y problema identificado

Durante el Sprint 3, el equipo detectó cuellos de botella recurrentes en la integración del módulo de autenticación (Auth-Frontend) con el backend, específicamente con el motor de agendamiento. Esto ocasionaba retrabajos (hasta 3 días de retraso en tareas dependientes) y fricción entre los desarrolladores frontend y backend.

5.4.2. Ciclo PDCA aplicado

- **Planificar:** Los responsables técnicos (Alex Guachi – frontend, Sebastián Ortiz – backend) propusieron un plan de mejora: (a) establecer revisiones de código conjuntas antes de cada integración, (b) implementar pruebas de integración automatizadas (usando Jest y Supertest), (c) realizar reuniones breves diarias de coordinación (15 minutos) para alinear contratos de API. El PM aprobó la estrategia.

- **Hacer:** Se implementaron las pruebas de integración que validaban el flujo completo de autenticación y creación de citas. Se ejecutaron sesiones de revisión de código en pares (frontend+backend) antes de fusionar ramas. Se estableció un canal de comunicación dedicado en Slack para resolver dudas inmediatas sobre contratos.

- **Verificar (Controlar):** El líder de proyecto realizó un seguimiento comparativo: se midió el número de incidencias relacionadas con integración antes y después de las medidas. Los reportes de sprint mostraron una reducción del 75 % en errores por desacoplamiento y una mejora del 40 % en el tiempo de entrega de tareas integradas. El feedback del equipo indicó una mayor fluidez en el trabajo colaborativo.

- **Actuar:** Se estandarizaron las prácticas efectivas: las pruebas de integración pasaron a ser obligatorias en el pipeline de CI/CD, las revisiones conjuntas se incorporaron como requisito para cualquier merge que involucre más de un subsistema, y las reuniones de coordinación se mantuvieron como parte del Daily Scrum. Los procesos que aún generaban pequeños retrasos (por ejemplo, la documentación de APIs) fueron ajustados con plantillas automáticas.

5.4.3. Resultados y evidencias

La tasa de errores de integración se redujo drásticamente, los sprints siguientes se completaron dentro de los plazos estimados y la moral del equipo mejoró. Las pruebas automatizadas pasaron a formar parte del estándar de calidad del proyecto.

5.5. Análisis transversal de los ciclos PDCA

La aplicación sistemática del PDCA permitió no solo corregir problemas puntuales sino también construir una cultura de mejora continua. Cada ciclo generó artefactos que alimentaron la base de conocimiento del proyecto: plantillas de requisitos, guías de modelado de datos, políticas de integración y pruebas automatizadas. Se observó que los problemas de fases tempranas (requisitos, arquitectura) tenían un impacto multiplicador en fases posteriores, por lo que la verificación temprana resultó crítica. El equipo recomienda mantener las matrices PDCA como parte de la documentación obligatoria al final de cada sprint.

6. Anexos

6.1. Anexo A: Matriz PDCA – Fase 1 (Planificación y requisitos)

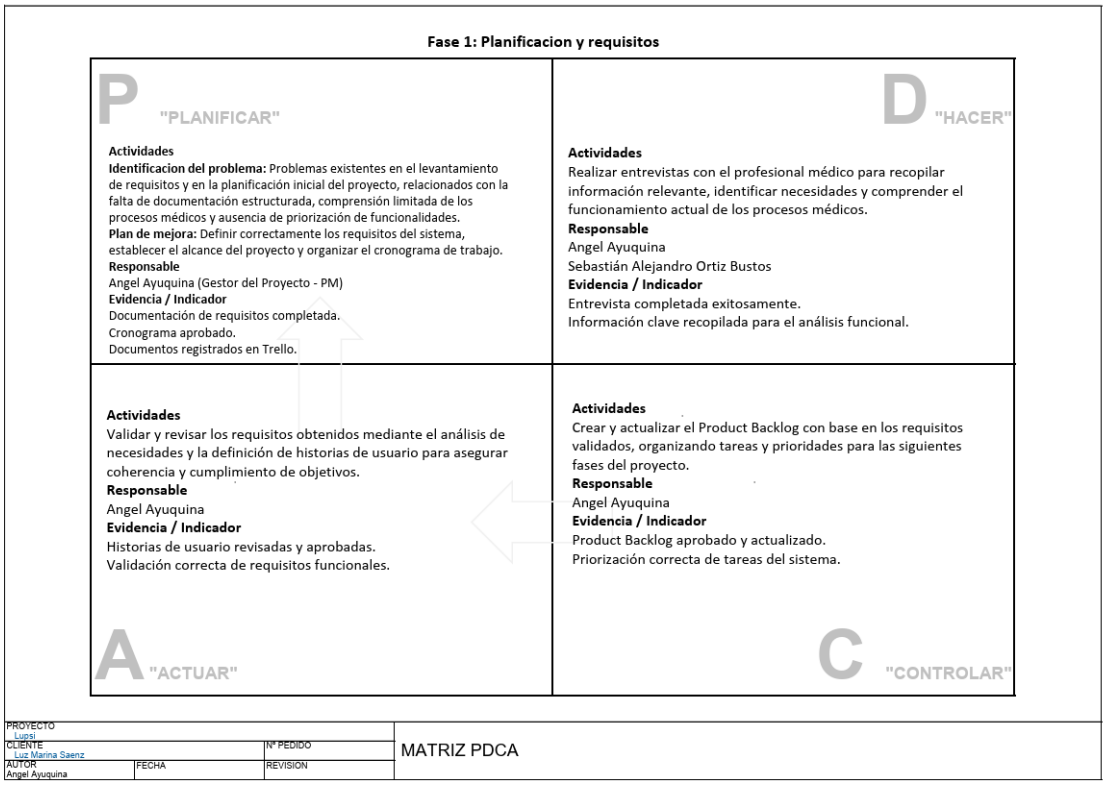


Figura 1: Matriz PDCA correspondiente a la Fase 1 de planificación y requisitos.

6.2. Anexo B: Matriz PDCA – Fase 2 (Arquitectura y Diseño UI/UX)

Fase 2 : Arquitectura y Diseño UI/UX

P "PLANIFICAR"

Problema Identificado:
Falta de un modelo de persistencia relacional normalizado y de prototipos de interfaces validados para el flujo médico-paciente en la nube. Esto genera un alto riesgo de redundancia de datos, retrasos de desarrollo por falta de un entorno colaborativo estándar,
Actividades:
Diseñar el diagrama Entidad-Relación (ER) conceptual y lógico para el control de citas.
Elaborar wireframes y flujos de navegación para el Portal del Paciente y Panel Administrativo.
Definir formalmente la arquitectura del sistema (Angular PWA <-> NestJS <-> Supabase)
Responsable:
Daniel Luisa (Arquitectura y Entornos locales), **Sebastián Ortiz** (Modelado lógico BD), **Alex Guachi** (Diseño UX/UI) y **Angel Ayuquina** (PM / Coordinador).
Evidencia / Indicador:
Product Backlog y requerimientos aprobados por el docente, especificación de tecnologías base definida, e historias de usuario mapeadas en el tablero.

D "HACER"

Actividades:
Construir el diagrama Entidad-Relación definitivo normalizado a la **Tercera Forma Normal (3FN)** y redactar el borrador del script de sentencias DDL (tablas profiles, patients, appointments, doctors).
Diseñar prototipos interactivos de alta fidelidad y navegables utilizando herramientas de diseño UX/UI.
Inicializar repositorios del proyecto en GitHub configurando el flujo de ramas bajo el modelo **GitFlow**.
Generar contenedores locales en Docker para estandarizar los entornos locales de desarrollo de los programadores.
Responsable:
Equipo de desarrollo (**Sebastián Ortiz**, **Daniel Luisa** y **Alex Guachi**).
Evidencia / Indicador:
Repositorio base inicializado en GitHub, esquemas DDL lógicos completados y prototipos navegables presentados en Draw.io/Lucidchart.

A "ACTUAR"

Actividades:
Realizar ajustes visuales a los prototipos de pantallas de agendamiento basándose en la retroalimentación de la Dra. Saenz.
Implementar reglas estrictas de integración colaborativa en GitHub, estableciendo **Pull Requests (PR)** y revisiones de código obligatorias para solucionar y prevenir conflictos de fusión.
Exportar las sentencias DDL definitivas y estructurar las políticas de **Row Level Security (RLS)** listas para la implementación física de las tablas en Supabase.
Ajustar y afinar la especificación de APIs para preparar la transición fluida hacia el desarrollo y codificación activa en el Sprint 2.
Responsable:
Todo el equipo de desarrollo y el Gestor del Proyecto (**Angel Ayuquina**).
Evidencia / Indicador:
Flujo GitFlow saneado (cero conflictos de fusión), base de datos lista para desplegarse físicamente, y diseños UX/UI definitivos.

C "CONTROLAR"

Actividades:
Realizar sesiones de validación y usabilidad de las interfaces diseñadas con los usuarios clave: **Dra. Luz Marina Saenz** (cliente) e **Ing. Paulo Torres** (patrocinador académico).
Auditar el modelo de datos diseñado en Draw.io para asegurar que cumpla rigurosamente con la integridad referencial y las reglas de normalización (3FN).
Verificar el cumplimiento de todos los entregables de la Fase 2 frente a la planificación inicial de Sprint 1.
Ejecutar revisiones internas del código DDL inicial y el diagrama lógico para evitar inconsistencias técnicas.
Responsable:
Angel Ayuquina (Líder del proyecto) y responsables de área.
Evidencia / Indicador:
Aprobación oficial con firmas digitales del **Informe de Modelado de BD y Arquitectura** (Código **MI-DES-ARQ-002**) con fecha **17/03/2026** por todo el equipo, y feedback positivo documentado.

Figura 2: Matriz PDCA de la Fase 2 de arquitectura y diseño de interfaces.

6.3. Anexo C: Matriz PDCA – Fase 3 (Desarrollo)

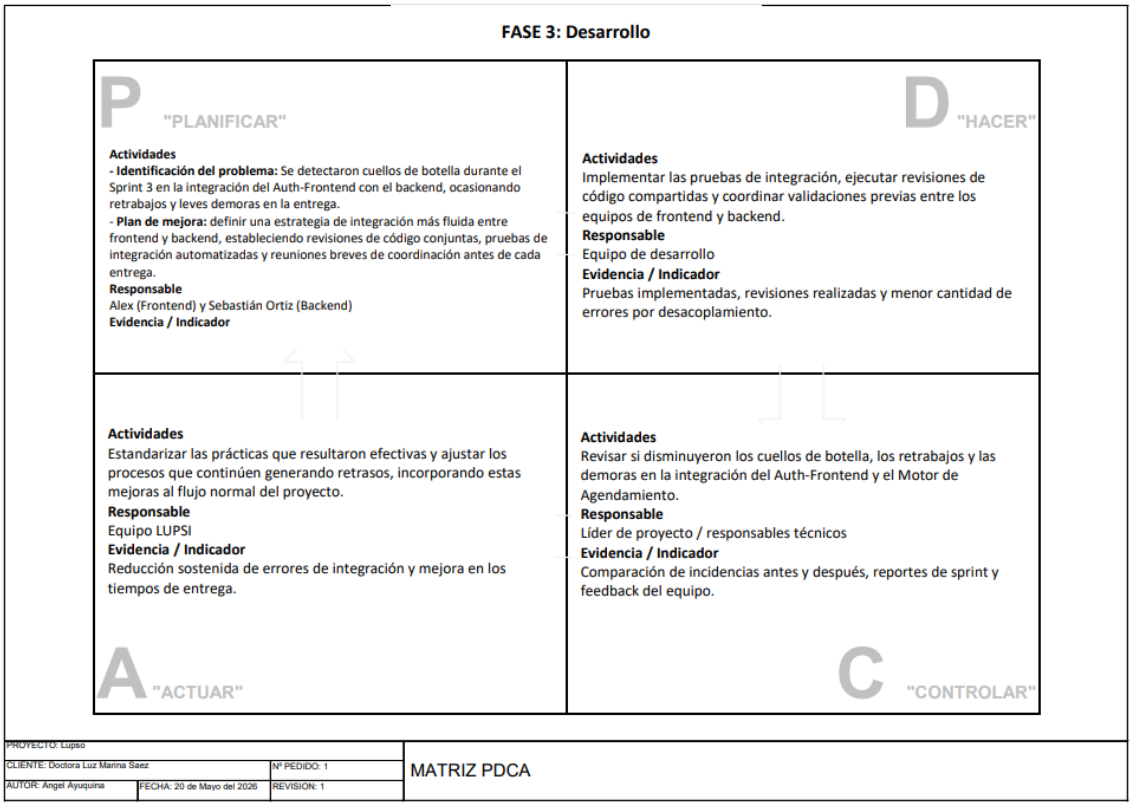


Figura 3: Matriz PDCA de la Fase 3 de desarrollo e integración.

SKT SOFTWARE SOLUTION	**SKT Software Solution**	**CÓDIGO:** DI-GDR-RGI-001
	PROCESO: Gestión de Proyectos - Ciclo PDCA	**VERSIÓN:** 1.0
	Informe de Mejora Continua	**ELABORACIÓN:** 21/05/2026
	Matrices PDCA del Proyecto Centro Médico	**ÚLTIMA REVISIÓN:** 21/05/2026

7. Firmas de Responsabilidad

Rol	**Nombre / Representante**	**Firma (QR)**	**Fecha**
Gestor del Proyecto	Angel Ayuquina		21/05/2026
Desarrollador Backend	Sebastián Ortiz		21/05/2026
Desarrollador Fullstack	Daniel Luisa		21/05/2026
Desarrollador Frontend	Alex Guachi		21/05/2026